

pyBE — Complete Master Ideas List

Every PR Idea, Fully Detailed, Ready to Implement

How to read this document: Every idea is written so that a developer reading it for the first time can understand what it is, why it exists, how to build it start-to-finish, which files to touch, which libraries to use, and how it fits into the pyBE manifesto. Ideas are grouped into three sections: Content PRs (philosophical learning modules), Platform Engineering PRs (technical infrastructure), and Community/Feature PRs (ecosystem features).

SECTION A — PHILOSOPHICAL LEARNING MODULE PRs

These are scenario-level content PRs. Each one teaches a Python concept through a non-programming domain. Every idea in this section must strictly follow the 4-pillar pyBE Manifesto framework: Theory → Anchor → Trigger → Reality. These are the PRs that get you recognized as a serious contributor.

A1 — "The Panchatantra Mirror"

Python Concept: Introspection, Reflection & Metaclasses Manifesto Level: 3–4 | Jonasan
Type: Structured Inquiry

What This Is

A complete learning module that teaches Python's self-inspection capabilities — the `inspect` module, `dir()`, `hasattr()`, `getattr()`, `__dict__`, and eventually metaclasses — using the ancient Panchatantra story of the clever rabbit and the lion as the philosophical anchor. The Panchatantra is explicitly listed in the mind map as an unclaimed domain under "Folklore," making this the first PR to claim it.

The Four Pillars

Why (Theoretical Anchor): The Panchatantra story "The Clever Rabbit and the Lion" is not a children's tale — it is a lesson in weaponizing self-awareness. The rabbit, physically outmatched, doesn't fight the lion. He shows the lion his own reflection in a well. The lion, seeing what appears to be a rival lion, charges and drowns himself. The rabbit wins by making his opponent confront its own image. This is the philosophical root of Python's introspection system: a program that can examine itself at runtime is a program that can outsmart rigid, static systems. The moment a Python object can look at itself and describe its own capabilities, it becomes as powerful as the rabbit.

Anchor (Interdisciplinary Mapping): Folklore (Panchatantra) → Reflection as tactical power → Python's `inspect` module and metaclass system. The rabbit's use of the well-mirror maps directly to Python's ability to examine its own objects using `dir()`, `type()`, `inspect.getmembers()`, and `__dict__` at runtime.

Trigger (Case Study — The Problem):

You are building a hospital's patient record processing system. This system receives patient objects from three different hospital branches. Each branch sends a different version of the Patient object: Branch A's version has `billing_code`, Branch B's version has `insurance_id`, and Branch C's version has both. At runtime, your processing engine must figure out what it's holding and decide how to handle it — without crashing if a field doesn't exist.

The constraint: you are NOT allowed to ask any hospital which version they sent. You cannot use `isinstance()`. You cannot use `try/except` as your primary logic. You must look at the object's own reflection.

How does a program examine itself?

This problem forces the learner, out of necessity, to discover: `dir(obj)` to list all attributes, `hasattr(obj, 'field')` to safely check existence, `getattr(obj, 'field', default)` to safely retrieve, `obj.__dict__` to see the raw attribute map, and finally `inspect.getmembers(obj)` for a complete inventory.

Reality (Software Engineering Depth): Demonstrate `functools.lru_cache` — it introspects its own call history at runtime to make eviction decisions. Then build a metaclass that auto-logs every method call on any class that inherits from it — this is a real production pattern used in ORMs, audit systems, and middleware. Show hardware tie-in: a CPU's branch predictor does the same thing — it maintains a history table of past branch outcomes and inspects that history to predict future execution paths. This is hardware-level introspection. Tie the metaclass pattern to how Django's ORM builds model fields by inspecting class definitions at class-creation time.

What to Build (Feature Specification)

Files to create:

```
server/src/scenarios/panchatantra-mirror/
├─ scenario.json          # Scenario metadata (title, domain, level, anchor)
├─ case-study.md          # The full narrative trigger with the hospital
problem
├─ hints.json             # 5 progressive hints, each nudging toward inspect
├─ expected-constructs.json # List of Python constructs the learner should
discover
├─ scoring-rubric.json    # How to score: reasoning (40), code (30), reflection
(30)
├─ solution/
│   └─ solution.py        # Reference solution using
dir/hasattr/getattr/inspect
├─ metaclass_demo.py      # The advanced metaclass auto-logger
└─ reflection-prompts.json # 4 reflection questions shown after submission
```

Libraries:

- `inspect` (stdlib) — core module being taught
- `functools` (stdlib) — for `lru_cache` demonstration
- No external dependencies needed for the scenario itself

Step-by-Step Implementation:

1. Write `scenario.json` with fields: `id`, `title`, `domain` ("Folklore/Panchatantra"), `philosophicalAnchor`, `pythonConcept`, `difficultyLevel` (3), `jonsasanType` ("Structured Inquiry"), `targetConstructs` (array)
2. Write `case-study.md` — the hospital scenario narrative, written in second person present tense. No Python syntax. No hints. Just the problem.
3. Write `hints.json` — 5 hints, each one nudging slightly further: Hint 1: "Can you ask the object to describe itself?" Hint 2: "Is there a built-in function that lists everything an object 'knows'?" Hint 3: "What if you could check whether a thing exists before you touch it?" etc.
4. Write `solution.py` — the full working solution demonstrating `dir()`, `hasattr()`, `getattr()`, `inspect.getmembers()`, then the metaclass
5. Write `reflection-prompts.json` — questions like: "Why is checking for existence before accessing safer than using try/except?", "What does it mean for a program to 'know about itself'?", "Where else in your life do you inspect before you act?"
6. Register the scenario in `server/src/data/scenarios.json`
7. Add the domain tag "Folklore" and "Panchatantra" to the tag taxonomy

PR Description Template:

Theoretical Anchor: Panchatantra's "Rabbit and Lion" – reflection as tactical intelligence
Interdisciplinary Mapping: Folklore → Self-Awareness → Python inspect module
Trigger Type: Structured Inquiry (Jonasan)
Constructs Taught: `dir()`, `hasattr()`, `getattr()`, `__dict__`, `inspect.getmembers()`, metaclass
Level: 3-4
Reality: metaclass auto-logger pattern used in production ORMs; CPU branch predictor as hardware parallel

A2 — "The DNA Helix"

Python Concept: Recursion, Call Stack, Stack Frames & Memory Limits Manifesto Level: 3–4 | Jonasan Type: Structured Inquiry

What This Is

A learning module that uses the structure of DNA — specifically Linus Pauling's discovery of the alpha-helix in 1951 — to teach recursion, call stacks, Python's recursion limit, and how to convert recursive algorithms to iterative ones for memory safety. Biology is listed in the mind map ("Protein Structures, ATCG") as a domain but has no PRs claiming it yet.

The Four Pillars

Why (Theoretical Anchor): DNA doesn't store genetic instructions as a flat linear string — it folds onto itself. A protein's function is determined not by its amino acid sequence alone, but by the shape that sequence folds into. Each fold references the shape of previous folds. This is structurally

recursive. Linus Pauling, studying protein structures in 1951, discovered the alpha-helix not by reading sequence data but by realizing that the physical shape was self-referential at every scale. Life itself uses recursion as its primary organizational strategy. A Python function that calls itself is not a programming trick — it is the most natural structure in biology.

Anchor (Interdisciplinary Mapping): Molecular Biology → DNA's hierarchical folding pattern → Python recursion. Each level of protein folding (primary → secondary → tertiary → quaternary structure) maps to a recursive call going deeper and deeper, with the base case being the innermost stable fold.

Trigger (Case Study):

A DNA sequencing lab is processing a gene. The gene is encoded in segments. Each segment, before it can be decoded, requires you to first decode a "template segment" from earlier in the strand. That template segment, in turn, requires another template.

You receive Segment A. To decode A, you need to first decode Segment B. To decode B, you first need Segment C. Segment C has no dependency — it is the ground truth.

Write a decoder. Your system must handle this chain of dependencies.

New constraint: the lab's sequencer has just detected a gene with 15,000 nested segment references. Your decoder crashes. Why? What does Python's memory have to do with gene length?

This forces discovery of: recursion with a base case, call stack growth, `sys.getrecursionlimit()` (default 1000), `RecursionError`, `sys.setrecursionlimit()` and why blindly raising it is dangerous (each frame = memory), and finally converting to an iterative approach using an explicit stack (`collections.deque`).

Reality (Software Engineering Depth): Use `tracemalloc` to demonstrate live memory growth per recursive call. Show exactly how many bytes each stack frame consumes. Demonstrate `sys.getrecursionlimit()` and `sys.setrecursionlimit()`. Then convert the recursive decoder to an iterative one using `collections.deque` as an explicit stack — show the memory profile is now flat. Hardware tie-in: CPUs have a hardware call stack (the return address stack buffer) with a fixed depth of ~16–30 entries. Buffer overflow attacks exploit what happens when this stack is exceeded. Python's software recursion limit exists for the same reason hardware stacks have limits.

What to Build

Files:

```
server/src/scenarios/dna-helix-recursion/
├── scenario.json
├── case-study.md           # The gene decoder narrative
├── hints.json             # 5 hints progressing toward recursion → base case →
limit → iterative
├── gene_data.json         # Sample gene segment data for the learner to work
with
├── solution/
│   ├── recursive_decoder.py  # Clean recursive solution
│   └── memory_profile.py     # tracemalloc demonstration of stack growth
```

```
└─ iterative_decoder.py    # The optimized iterative version with deque
└─ reflection-prompts.json
```

Key code the solution must demonstrate:

```
import sys
import tracemalloc
from collections import deque

# 1. Recursive version (elegant but limited)
def decode_segment(segment_id, gene_map):
    segment = gene_map[segment_id]
    if segment['dependency'] is None: # Base case
        return segment['data']
    dependency_data = decode_segment(segment['dependency'], gene_map)
    return decode(segment['data'], dependency_data)

# 2. Show the crash
sys.setrecursionlimit(10) # Force early failure to demonstrate

# 3. Iterative version (memory-safe)
def decode_iterative(start_id, gene_map):
    stack = deque()
    current = start_id
    while gene_map[current]['dependency'] is not None:
        stack.append(current)
        current = gene_map[current]['dependency']
    result = gene_map[current]['data']
    while stack:
        seg_id = stack.pop()
        result = decode(gene_map[seg_id]['data'], result)
    return result
```

A3 — "The Fellowship's Graph"

Python Concept: Graph Theory, Pathfinding & Dijkstra's Algorithm Manifesto Level: 4 |
Jonasan Type: Dilemma (no single correct answer)

What This Is

A learning module using Tolkien's "The Lord of the Rings" — specifically the Fellowship's route decision at Caradhras/Moria — to teach graph representation, weighted edges, priority queues, and Dijkstra's shortest path algorithm. "Lord of the Rings" is listed in the mind map under Pop Culture Integration but has no implementation yet.

The Four Pillars

Why (Theoretical Anchor): Tolkien's Fellowship does not choose their route randomly. At Caradhras, Saruman blocks the mountain pass with a storm — the edge weight of that route suddenly increases to near-infinity. The Fellowship is forced to reconsider. Moria (Mines of Khazad-dûm) is shorter in distance but carries a different kind of cost: danger, darkness, and unknown threats. Gandalf chooses Moria knowing the "distance" is shorter but the "risk cost" is high and uncertain. This is exactly what Dijkstra's algorithm does — it continuously re-evaluates paths as new information about edge costs becomes available, seeking the minimum total cost path, where "cost" can be any quantifiable trade-off, not just distance.

Anchor (Interdisciplinary Mapping): Literature (Tolkien/LOTR) → Strategic route planning under dynamic constraints → Weighted graph traversal and Dijkstra's algorithm. The key insight: edge weights in real graphs are not always distance. They can be risk, time, moral cost, resource expenditure — any quantifiable trade-off.

Trigger (Case Study):

A disaster relief logistics coordinator must route emergency medical supplies from a central depot (Mumbai) to a flood-affected village (Kolhapur). There are 7 possible routes, each with two costs: travel time (in hours) and risk score (bridges may be damaged, roads may be flooded — each has a probability of failure).

Route 3 is fastest at 4 hours but has a 60% chance of a bridge collapse. Route 6 is safe but takes 9 hours. Route 2 is 6 hours and has a 20% risk.

Question 1: How do you represent this network as a data structure? Question 2: How do you find the path that minimizes total travel time? Question 3: How do you find the path that minimizes total risk? Question 4: How do you find a path that balances both? Is there a single "correct" answer?

Question 4 makes this a Dilemma (Jonasan type) — there is no objectively correct answer. The learner must make a value judgment about how to weight competing costs. This is where Tolkien comes back in: Gandalf made a moral judgment, not a mathematical one.

Reality (Software Engineering Depth): Implement the graph as an adjacency dictionary. Implement Dijkstra's with `heapq` (Python's built-in min-heap). Show the time complexity: $O((V + E) \log V)$. Demonstrate with `timeit` how performance degrades on a dense vs sparse graph. Hardware tie-in: CPU schedulers use priority queues (Completely Fair Scheduler in Linux) to manage process execution — the same data structure (`heapq`) underlies both the Fellowship's route and the Linux kernel's process scheduler.

What to Build

Files:

```
server/src/scenarios/fellowship-graph/
├── scenario.json
├── case-study.md
├── network_data.json      # The 7-route network with time and risk costs
├── hints.json
├── solution/
│   ├── graph_basics.py    # How to represent the network as a dict
│   ├── dijkstra.py        # Full Dijkstra's implementation using heapq
│   ├── multi_objective.py # The "balance both costs" version
│   └── complexity_demo.py # timeit showing  $O((V+E)\log V)$  in practice
└── reflection-prompts.json
```

Key implementation the solution must include:

```
import heapq

def dijkstra(graph, start, cost_key='time'):
    # graph = {'Mumbai': {'Pune': {'time': 2, 'risk': 0.1}, ...}, ...}
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    priority_queue = [(0, start)]
```

```

previous = {}

while priority_queue:
    current_cost, current_node = heapq.heappop(priority_queue)
    for neighbor, edge_data in graph[current_node].items():
        cost = edge_data[cost_key]
        new_cost = current_cost + cost
        if new_cost < distances[neighbor]:
            distances[neighbor] = new_cost
            previous[neighbor] = current_node
            heapq.heappush(priority_queue, (new_cost, neighbor))
return distances, previous

```

A4 — "The Nashika Taal Engine"

Python Concept: Loops, Generator Functions, Lazy Evaluation & Threading Manifesto

Level: 3–4 | Jonasan Type: Structured Inquiry

What This Is

This is the PR that fully implements the Nashik Dhol connection that the manifesto explicitly mentions but no contributor has yet built. It uses the taal (rhythmic cycle) of Nashik Dhol percussion to teach the philosophical difference between a `for` loop (pre-computed, memory-heavy) and a generator function (computed on demand, memory-light), then escalates to threading and the GIL.

The Four Pillars

Why (Theoretical Anchor): A master Nashik Dhol player does not memorize every individual beat of a 10,000-beat performance. They internalize the rule that generates the pattern. The taal (rhythmic cycle, like a 16-beat Teentaal) is the formula, not the sequence. The player generates each beat on demand, in the moment, using the rule. This is the philosophical heart of Python's generator functions: instead of pre-computing and storing all values in memory (like writing out all 10,000 beats on paper), a generator stores only the rule and computes each value lazily, on demand. `range(10000)` generates nothing in advance. The rule is the data structure.

Anchor (Interdisciplinary Mapping): Indian Classical Music → Nashik Dhol → Taal (rhythmic formula) → Generator functions (`yield`) as lazy, rule-driven computation. The synchronization of multiple dhol players maps to Python threading.

Trigger (Case Study):

A music system must simulate three Nashik Dhol players performing simultaneously: - Player 1 strikes every 2 beats - Player 2 strikes every 3 beats - Player 3 strikes every 5 beats

Task 1: After how many beats does everyone strike at the same moment? Task 2: Now simulate this for 1,000,000 beats. Generate every synchronization point. Your system has only 64MB of RAM.

If you generate a list of all 1,000,000 beats, you run out of memory. How do you generate the sequence without ever storing it?

Task 3: The three players are in different locations and must play simultaneously via a network. How do you coordinate them in code?

Task 1 teaches: LCM (Least Common Multiple), `math.lcm()`. Task 2 forces the discovery of generators (`yield`) — the learner cannot use a list for 1M items in 64MB. Task 3 introduces `threading.Thread` and `threading.Event` for synchronization.

Reality (Software Engineering Depth): Show memory comparison using `sys.getsizeof()`: a list of 1M integers vs a generator object (the list is megabytes; the generator is ~112 bytes). Demonstrate Python's GIL and why `threading` doesn't give true CPU parallelism for compute-bound tasks. Introduce `multiprocessing` as the true parallelism tool. Hardware tie-in: audio hardware uses ring buffers — circular buffers with a fixed size that overwrite old data. Implement one using `collections.deque(maxlen=n)` — this is how audio drivers manage sample buffers at the firmware level.

What to Build

Files:

```
server/src/scenarios/nashika-taal/
├── scenario.json
├── case-study.md
├── hints.json
├── solution/
│   ├── list_version.py          # The naive list version – shows memory
└── explosion
    ├── generator_version.py     # The generator solution with yield
    ├── memory_comparison.py    # sys.getsizeof() comparison
    ├── threading_demo.py       # Three threads synchronized with Event
    ├── ring_buffer.py          # Hardware-parallel: deque ring buffer
└── reflection-prompts.json
```

A5 — "Newton's Prism & The Bit Painter"

Python Concept: Bitwise Operators, Hexadecimal & Binary Memory

Representation Manifesto Level: 3–4 | Jonasan Type: Structured Inquiry

What This Is

A module using Isaac Newton's prism experiment and Color Theory (listed in the mind map under Philosophical Frameworks) to teach bitwise operators — AND (&), OR (|), XOR (^), left shift (<<), right shift (>>) — and binary/hexadecimal number representation. The philosophical insight is that Newton didn't add color to white light — he revealed what was already there. Similarly, bitwise operators don't create data — they reveal structure that was always inside the number.

The Four Pillars

Why (Theoretical Anchor): When Isaac Newton passed white light through a prism in 1666, he discovered that white light already contained all colors. The prism was not a creative tool — it was a revealing tool. It separated what was always there. This is the exact philosophy of bitwise operations: a 32-bit integer doesn't hold "one number" — it holds 32 independent binary states, each

waiting to be extracted with the right mask. Hexadecimal notation (0xFF8800) is not a different number — it is the same number viewed through a different prism, one that makes the byte boundaries visible to human eyes.

Anchor (Interdisciplinary Mapping): Color Theory & Physics (Newton's Optics) → Decomposition of compound wholes into component parts → Bitwise masks as computational prisms that reveal hidden structure within integers.

Trigger (Case Study):

You are a digital archivist restoring a collection of 1980s digital artworks. The original artist stored an entire RGB color in a single 32-bit integer using a compact format from 1987: - Bits 24–31: Alpha channel (transparency) - Bits 16–23: Red channel - Bits 8–15: Green channel - Bits 0–7: Blue channel

Example: the integer 0xFFCC6600 represents Alpha=255, Red=204, Green=102, Blue=0.

Your task: build a color decoder. Given any packed integer, extract all four channels.

Rules: No string parsing. No converting to binary strings. No splitting. You must work with the integer itself using mathematical operations.

Bonus: Build an encoder that takes R, G, B, A values and packs them back into a single integer.

This forces discovery of: hex literals (0xFF), right shift (>>) to move bits into position, AND mask (& 0xFF) to isolate the last 8 bits, left shift (<<) for encoding, OR (|) for combining channels.

Reality (Software Engineering Depth): Python's `struct` module for packing/unpacking raw bytes — used in network protocol implementations, binary file parsers, and hardware interfaces. Live demo: open and parse the first 14 bytes of an actual BMP file header. The BMP header contains image width, height, and color depth packed as bitfields — this is genuine file format reverse engineering at the binary level. Hardware tie-in: MMIO (Memory-Mapped I/O) registers in microcontrollers are controlled exactly this way — a single 32-bit register controls multiple hardware features through bitmasks. Show a GPIO control register example.

What to Build

Files:

```
server/src/scenarios/newtons-prism/
├── scenario.json
├── case-study.md
├── artwork_palette.json  # Sample packed color integers for the learner
├── hints.json
├── solution/
│   ├── bit_decoder.py    # The color extractor using bitwise ops
│   ├── bit_encoder.py    # The color packer
│   ├── struct_demo.py    # struct module for BMP header parsing
│   └── sample.bmp        # A tiny BMP file for live parsing demo
└── reflection-prompts.json
```

A6 — "The Buddhist Garbage Collector"

Python Concept: Variable Scope, Reference Counting, Circular References & weakref Manifesto Level: 4 | Jonasan Type: Structured Inquiry

What This Is

The manifesto references Buddhism as an anchor for variable existence but doesn't fully operationalize it. This PR goes deeper: Buddhist philosophy of Anicca (impermanence) explains not just why variables should have purpose, but why they must eventually cease. Python's garbage collector is, philosophically, the principle of Anicca made computational. A variable that clings to memory beyond its useful life causes suffering (memory leaks). The `weakref` module is the Buddhist practice of non-attachment made into code.

The Four Pillars

Why (Theoretical Anchor): Anicca — the Buddhist concept of impermanence — states that all conditioned phenomena arise, persist for a time, and must cease. Clinging (*upadana*) to what must cease causes suffering (*dukkha*). In software, an object that cannot be garbage collected because some reference clings to it beyond its useful life is not just wasteful — it is a system in suffering (memory leak, degraded performance, eventual crash). Python's reference-counting garbage collector embodies Anicca mechanically: when an object's reference count drops to zero, it ceases — it does not "die," it simply stops being held. The `weakref` module allows you to reference an object without preventing its natural cessation — the software equivalent of non-attachment.

Anchor (Interdisciplinary Mapping): Buddhist Philosophy → Anicca (Impermanence) + Upadana (Clinging) → Python's `gc` module, reference counting, circular references as clinging, `weakref` as non-attachment.

Trigger (Case Study):

A hospital patient management system creates a `Patient` object and a `Ward` object. When a patient is admitted, the ward adds the patient to its roster (`ward.patients.append(patient)`). The patient also keeps a reference to their ward (`patient.ward = ward`).

When the patient is discharged, the system does:

```
del patient
del ward
```

Both variables are deleted. Memory should be freed. But your memory profiler shows the memory is never freed. The patient and ward are still "alive" in memory even though no one is holding them.

Why? What is preventing the garbage collector from releasing them?

How do you model a relationship between two objects where each knows about the other, but where destroying one does not trap the other in memory forever?

This forces discovery of: circular references ($A \rightarrow B, B \rightarrow A$ means neither reaches zero), `sys.getrefcount()` to count references, `gc.collect()` to manually trigger collection,

`gc.get_referrers()` to find what's holding an object, and finally `weakref.ref()` and `weakref.WeakValueDictionary` as the solution.

Reality (Software Engineering Depth): Live demonstration using `tracemalloc` to prove the memory leak. Manual `gc.collect()` call showing it can resolve cycles but at a performance cost. Show the `__del__` method as an object's "last rites" — and why it is dangerous in circular reference scenarios. Hardware/embedded tie-in: MicroPython on a Raspberry Pi Pico has no OS-level memory recovery. A circular reference in an embedded loop is fatal — the device will silently corrupt or freeze. Show the `weakref.WeakValueDictionary` as the production pattern for object registries.

What to Build

Files:

```
server/src/scenarios/buddhist-garbage-collector/
├── scenario.json
├── case-study.md
├── hints.json
├── solution/
│   ├── circular_ref_demo.py      # Demonstrates the memory leak
│   ├── refcount_inspection.py    # sys.getrefcount() exploration
│   ├── tracemalloc_proof.py      # Live memory leak proof
│   ├── weakref_solution.py       # The non-attachment fix
│   └── embedded_context.py       # MicroPython simulation (pure Python)
└── reflection-prompts.json
```

A7 — "Maslow's Stack — The Architecture of Needs"

Python Concept: Software Architecture Layers, ctypes, Hardware Interaction via sysfs
Manifesto Level: 5 (Hardware Touch) | Jonasan Type: Design Thinking Problem

What This Is

This is one of the most ambitious content PRs and the one most likely to hit Level 5 of the pyBE scale. It uses Maslow's Hierarchy of Needs (listed in the mind map under Philosophical Frameworks) as a literal architectural diagram: the UI is "self-actualization," the OS is "safety," and the hardware register is "physiological need." Just as humans cannot self-actualize without their physiological needs being met, a beautiful Python dashboard UI cannot function without the hardware sensor it reads from. This PR makes the learner trace a dependency chain all the way from the Python frontend down to an actual hardware register.

The Four Pillars

Why (Theoretical Anchor): Maslow's hierarchy is not just about human motivation — it is a model of dependency ordering. A person cannot pursue beauty and creativity (top of pyramid) if their survival needs (bottom) are unmet. The hierarchy is a statement about which layers must be satisfied before higher layers can function. Software architecture is Maslow's pyramid literalized: a beautiful React UI (self-actualization) depends on a working API (esteem), which depends on a stable OS process (belonging), which depends on kernel drivers (safety), which depends on physical hardware (physiological). Remove the bottom layer and the top collapses.

Anchor (Interdisciplinary Mapping): Psychology (Maslow's Hierarchy) → Dependency ordering in human needs → Layered software architecture. Each layer of the software stack maps to a level of the pyramid, and the Python `ctypes` module is the bridge that allows top-layer Python code to reach down to the physiological (hardware) layer.

Trigger (Case Study):

You are building a server room monitoring dashboard. A designer has given you a beautiful React mockup. A backend engineer has given you a REST API spec. But the data source is a hardware temperature sensor in the server — a raw Linux kernel interface at `/sys/class/thermal/thermal_zone0/temp`.

Start from the top (the UI). Trace every dependency downward. At what layer does your Python code first touch something that is not Python? Can Python directly read a hardware register? If so, how? Should it?

Build the complete chain: React component → Flask API endpoint → Python `sysfs` reader → OS kernel interface → hardware register.

Reality (Software Engineering Depth): Actually read `/sys/class/thermal/thermal_zone0/temp` using Python's file I/O (Linux only — this reads a hardware register exposed via the `sysfs` virtual filesystem). Demonstrate `ctypes.cdll.LoadLibrary('libc.so.6')` to call a C library function directly from Python — this is the `ctypes` bridge between Python and C/hardware. Build a complete Flask endpoint that reads the temperature and serves it to a minimal HTML dashboard. Show how this changes on different OSes (Windows uses WMI, macOS uses `ioreg`). This satisfies the Level 5 requirement: "firmware and hardware optimization where the software touches the physical world."

What to Build

Files:

```
server/src/scenarios/maslows-stack/
├── scenario.json
├── case-study.md           # The server room monitoring problem
├── architecture-diagram.md # Maslow pyramid mapped to software layers
├── hints.json
├── solution/
│   ├── hardware_reader.py # sysfs temperature reader (Linux)
│   ├── ctypes_demo.py     # ctypes calling libc directly
│   ├── flask_api.py       # REST endpoint serving hardware data
│   ├── dashboard.html     # Minimal HTML/JS frontend
│   ├── cross_platform.py  # Windows/macOS alternatives
└── reflection-prompts.json
```

A8 — "The Sorting Hat"

Python Concept: Classification Algorithms, Decision Trees & Binary Search Manifesto Level: 2–3 | Jonasan Type: Structured Inquiry

What This Is

The manifesto explicitly uses the Hogwarts Sorting Hat as an analogy for classification. This PR builds that analogy into a complete learning module. The Sorting Hat doesn't read minds randomly — it applies a rule set to features of each student to assign them to a house. This is exactly what a decision tree does. The PR teaches classification logic, the concept of features and labels, decision tree logic, and eventually Python's `bisect` module for binary search — all through the lens of Harry Potter.

The Four Pillars

Why (Theoretical Anchor): The Sorting Hat is an 11th-century classification algorithm with a training dataset of 1000 years of Hogwarts students. It doesn't sort randomly — it evaluates features (bravery, cunning, loyalty, intelligence) against a learned model and produces a label (Gryffindor, Slytherin, Hufflepuff, Ravenclaw). The critical insight: the Hat is a binary decision tree. At each node, it asks a yes/no question about a feature. Based on the answer, it branches left or right until it reaches a leaf node (a house). This is the fundamental structure of classification.

Anchor (Interdisciplinary Mapping): Pop Culture (Harry Potter) → The Sorting Hat as a classification oracle → Decision trees, feature evaluation, and binary search as classification mechanisms.

Trigger (Case Study):

You are rebuilding the Sorting Hat as a Python program for Hogwarts' digital enrollment system. You have a dataset of 200 past students with their personality scores (bravery: 0–10, cunning: 0–10, loyalty: 0–10, intelligence: 0–10) and the house they were sorted into.

Task 1: Given a new student's scores, determine which house to assign them. How do you compare? Task 2: 3000 students are enrolling simultaneously. For each, you need to quickly find the most "similar" past student to use as a reference. How do you search efficiently? Task 3: The Hat sometimes reconsiders. A student in Slytherin can appeal. How does your system handle re-classification without re-evaluating the entire dataset?

Task 1 teaches basic comparison and scoring. Task 2 forces discovery of sorting and `bisect` (binary search is $O(\log n)$ vs linear $O(n)$). Task 3 introduces the idea of incremental updates.

Reality (Software Engineering Depth): Implement both linear search and binary search with `bisect.insort()` and `bisect.bisect_left()`. Benchmark with `timeit` on 10,000 students. Show that $O(\log n)$ vs $O(n)$ means binary search is ~13x faster on 10,000 items. Introduce the concept of a simple hand-coded decision tree (without scikit-learn) to show the algorithmic structure clearly before any library is used.

A9 — "The Horcrux Pattern"

Python Concept: Distributed State, Serialization, JSON/Pickle & Database

Persistence Manifesto Level: 3 | Jonasan Type: Design Thinking Problem

What This Is

The manifesto explicitly maps Horcruxes to "distributed state or multiple stored parts." This PR builds that metaphor into a full lesson on Python serialization — `json`, `pickle`, `shelve` — and the concept of persisting application state across program restarts. A Horcrux is not magic — it is a distributed backup system. Voldemort sharded his soul across objects for fault tolerance. This is the philosophical foundation of distributed persistence.

The Four Pillars

Why (Theoretical Anchor): Voldemort's strategy was not evil vanity — it was an engineering decision about fault tolerance and persistence. By fragmenting his soul across 7 Horcruxes, he ensured that destroying any one node would not destroy the whole. This is the principle behind distributed databases, RAID systems, and any fault-tolerant data architecture. The question "where do you store state so it survives beyond the life of a single process?" is exactly the question Voldemort answered — and exactly the question any Python developer building a stateful application must answer.

Trigger (Case Study):

You are building a long-running Python research tool that analyzes academic papers. The tool takes 6 hours to process a full dataset. Halfway through, the power goes out.

When the program restarts, it starts from zero.

How do you make the program remember what it has already processed? How do you store its "progress soul" so it can survive a crash and resume?

Secondary challenge: You realize the state has grown to 500MB. Using Python's `json` module, saving takes 45 seconds. How do you make it faster?

This drives discovery of `json.dump()/json.load()`, `pickle.dump()/pickle.load()`, `shelve` for key-value persistent stores, checkpoint patterns (save progress at intervals), and comparing serialization performance: `json` vs `pickle` vs `marshal`.

A10 — "The Infinity Stones — Module Composition"

Python Concept: Python Packages, Modules, `__init__.py`, Import System & Namespace
Design Manifesto Level: 2–3 | **Jonasan Type:** Design Thinking Problem

What This Is

The manifesto maps the Infinity Stones to "composing modules into a full system." Each stone has one power; together they create omnipotence. Each Python module has one responsibility; together they create an application. This PR teaches proper package and module design — `__init__.py`, relative vs absolute imports, namespace packages, circular import avoidance — through the lens of the Gauntlet architecture.

Trigger (Case Study):

A team of 6 developers is building a fraud detection system. Each developer owns one capability: DataLoader, Preprocessor, Anomaly Detector, Risk Scorer, Alert System, Report Generator.

Currently all 6 capabilities are in one giant file: `fraud.py`. When one developer changes their code, they break everyone else's.

How do you separate these 6 capabilities so each developer works independently, yet the system still functions as one unified tool? How do you design the "Gauntlet" (the main entry point) that assembles all six?

A11 — "The Banana Leaf Feast — Spatial Memory Management"

Python Concept: Custom Memory Allocator, ctypes, Heap vs Stack & Memory

Paging Manifesto Level: 5 (Hardware/OS level) | Jonasan Type: Design Thinking Problem

What This Is

From the second source document. Uses the strict spatial discipline of a traditional South Indian meal served on a banana leaf (ragi mudde, sambar, rasam — each has a designated section that must not be breached) as the anchor for teaching memory segmentation, heap vs stack allocation, and building a custom memory allocator using Python's `ctypes` to request raw memory from the OS.

The Four Pillars

Why (Theoretical Anchor): Gestalt Principles of Grouping + Spatial Sociology: order is maintained only when discrete elements are given strict, segregated boundaries to prevent systemic chaos. A traditional feast on a banana leaf fails if liquids breach solid boundaries — the structure of the meal collapses. RAM is exactly this: strict spatial boundaries between stack (solid, fixed-size, structured), heap (liquid, dynamic, variable-size), and code segments. If they overlap, the system crashes. Memory corruption is the banana leaf with no borders.

Anchor (Interdisciplinary Mapping): Traditional Culinary Architecture (South Indian feast on banana leaf) → Spatial discipline and segregated boundaries → Memory segmentation: heap vs stack vs code segments.

Trigger (Case Study):

You have a strictly bounded rectangular space (128MB of RAM). You need to store two types of data simultaneously: - "Liquid" data: a list of user sessions that constantly grows, shrinks, and changes size - "Solid" data: a set of configuration constants that never change size after startup

If the liquid data expands into the solid data's space, the system crashes.

How does a computer organize these two fundamentally different types of data? Who decides where each goes? Can Python control this directly?

Reality (Software Engineering Depth): Use `ctypes` to call `malloc()` from `libc` directly — request raw, unformatted memory blocks from the OS. Show the difference between stack

allocation (automatic, function-scoped) and heap allocation (manual, persistent). Build a minimal custom memory pool that manages a fixed-size block using Python. Demonstrate memory alignment requirements. Show what a segmentation fault actually is at the pointer level.

Key code the solution explores:

```
import ctypes

# Request raw memory from the OS (C's malloc, called from Python)
libc = ctypes.cdll.LoadLibrary("libc.so.6")
libc.malloc.restype = ctypes.c_void_p
libc.malloc.argtypes = [ctypes.c_size_t]
libc.free.argtypes = [ctypes.c_void_p]

# Allocate 1024 bytes directly
raw_memory = libc.malloc(1024)
print(f"Raw memory address: {hex(raw_memory)}")

# Build a simple memory pool tracker
class MemoryPool:
    def __init__(self, size):
        self.pool = (ctypes.c_byte * size)()
        self.offset = 0
        self.size = size

    def allocate(self, n_bytes):
        if self.offset + n_bytes > self.size:
            raise MemoryError("Pool exhausted – banana leaf is full")
        address = ctypes.addressof(self.pool) + self.offset
        self.offset += n_bytes
        return address
```

A12 — "The Combustion Engine — Concurrent Thread Scheduling"

Python Concept: Async Event Loops, `asyncio`, GIL Bypass & Thread Scheduling Manifesto

Level: 5 | **Jonasan Type:** Dilemma

What This Is

From the second source document. Uses the simultaneous subsystem coordination of a high-compression motorcycle engine (fuel injection, spark timing, exhaust valve clearances operating in overlapping synchrony) to teach Python's `asyncio` event loop, the Global Interpreter Lock (GIL), thread affinity, and `async/await` patterns.

The Four Pillars

Why (Theoretical Anchor): Laws of Thermodynamics and Mechanical Synchrony: a combustion engine cannot operate sequentially. If the cooling system waits for the fuel injector to finish before running, the engine overheats and seizes. Real-world systems are inherently concurrent. The engine is a physical proof that sequential thinking fails at scale. Python's `asyncio` event loop is the software equivalent of an engine control unit (ECU) — a single-threaded coordinator that interleaves multiple tasks so efficiently that they appear simultaneous.

Trigger (Case Study):

A drone control system manages 4 subsystems simultaneously: camera feed processing, GPS location updates, battery monitoring, and obstacle detection.

The camera needs a new frame every 33ms. GPS updates every 100ms. Battery checks every 500ms. Obstacle detection runs continuously.

If any subsystem blocks while waiting, another subsystem misses its deadline. A missed obstacle detection deadline can cause a crash.

How do you run 4 tasks "at the same time" in Python when Python is fundamentally single-threaded?

Reality (Software Engineering Depth): Implement all 4 subsystems as `async` coroutines with `asyncio.gather()`. Show with `time.perf_counter()` that they interleave efficiently. Explain the event loop's role. Demonstrate the GIL: why `threading.Thread` with CPU-bound tasks doesn't give speedup. Show `concurrent.futures.ProcessPoolExecutor` as the true parallelism escape hatch. Hardware tie-in: implement mutex locks with `asyncio.Lock()` and show how core affinity (`os.sched_setaffinity()` on Linux) binds threads to specific CPU cores — real-time embedded system control technique.

A13 — "The Kannada Poet — Semantic Pointer Architecture"

Python Concept: HashMap Implementation, LRU Cache & Register-Level Memory

Optimization Manifesto Level: 4–5 | Jonasan Type: Structured Inquiry

What This Is

From the second source document. Uses the challenge of translating culturally dense Kannada poetry into English — where a single Kannada word may carry three contextual meanings, making word-by-word duplication bloated and lossy — as the anchor for teaching hash maps, the LRU cache, and pointer/reference-based memory optimization.

The Four Pillars

Why (Theoretical Anchor): The Sapir-Whorf Hypothesis (Linguistic Relativity) states that the language a person speaks shapes their cognitive perception of reality. When translating deeply rhythmic, culturally dense regional poetry — like classical Kannada *kavya* — into English, a single word can encode layered semantic meaning. Duplicating the entire cultural context for every occurrence of the word is bloat. A skilled translator creates a semantic pointer: they define the meaning once ("this word carries echoes of X, Y, Z") and reference that definition throughout. Python's hash map is exactly this: hash the key once, store the meaning at the memory address, and reference it from everywhere via pointer — not by copying.

Trigger (Case Study):

A linguistic research tool is processing 10 million words from a corpus of classical Kannada poetry. Each word has a semantic entry: its roots, cultural context, emotional weight, and usage examples.

If you store a copy of the full semantic entry with every occurrence of the word, the dataset is 40GB. You have 4GB of RAM.

How do you store each unique semantic meaning exactly once and reference it from every occurrence without duplicating the data?

Bonus: The tool shows the 100 most recently accessed words in a side panel. The panel must update instantly. How do you cache these efficiently so the 101st lookup doesn't push out the 1st if it was just accessed again?

The main challenge forces discovery of: dict as a hash map, how Python's dict works internally (hash tables, collision resolution). The bonus forces discovery of `functools.lru_cache` and then building an LRU cache from scratch using `collections.OrderedDict`.

Reality (Software Engineering Depth): Build an LRU cache from scratch using `collections.OrderedDict` (the `O(1)` `move_to_end()` operation). Show that `functools.lru_cache` is the production version. Demonstrate cache hit/miss rates with a frequency counter. Hardware tie-in: this is exactly how CPU L1/L2/L3 caches work — the LRU eviction policy is implemented in hardware transistors. Show that Python's `lru_cache` maxsize maps to a CPU cache's associativity — the same trade-off between speed and size that CPU engineers make.

A14 — "The Badminton Survivor — Neural Pruning"

Python Concept: Matrix Sparsity, Weight Thresholding, CUDA Bindings & Tensor Memory Allocation Manifesto Level: 5 (Advanced Hardware/GPU) | Jonasan Type: Structured Inquiry

What This Is

From the second source document. Uses athletic kinetics — specifically the strategic movement economy of a badminton player exhausted in the final set — as the anchor for teaching neural network model compression: matrix sparsity, soft thresholding, and explicit GPU tensor memory management via Python-C/CUDA bindings. This is the most technically advanced content PR.

The Four Pillars

Why (Theoretical Anchor): The Principle of Least Action in physics states that nature always finds the path requiring the minimal expenditure of energy — not the shortest path, not the most elegant, but the minimally sufficient one. When a badminton player is exhausted in the final set, survival depends on ruthlessly eliminating all unnecessary movement. Every step that doesn't earn a point is waste. The player performs a real-time "structural pruning" of their movement repertoire — dropping flashy footwork, committing only to essential movements. A neural network under memory constraints must perform the same operation: identify and eliminate "lazy nodes" (weights close to zero that contribute little to output) while preserving the "movements that win points" (weights that carry significant signal).

Trigger (Case Study):

A visual inspection system running on an edge device (NVIDIA Jetson Nano, 4GB RAM) is detecting defects in silicon wafers at a semiconductor fab. The model is accurate (97%) but too slow — latency is 340ms per image, but the production line requires under 50ms.

The model has 85 million parameters. After profiling, you discover that 70% of the weight values are between -0.01 and 0.01 — they contribute almost nothing to the output.

"The network is exhausted and out of breath. How do you identify and drop the lazy nodes without blinding the system?"

Target: reduce parameters by 60% with less than 2% accuracy loss.

Reality (Software Engineering Depth): Implement soft-thresholded adaptive rank pruning from scratch in Python. Write C extension bindings using `ctypes` or `cffi` to explicitly manage tensor matrix allocations on GPU memory. Demonstrate on a small PyTorch model without using any pruning library. Show memory before/after with `torch.cuda.memory_summary()`. Compare latency before and after. This is the most advanced PR possible in pyBE — it genuinely requires deep reasoning that "basic AI" cannot generate.

A15 — "The Social Exchange API"

Python Concept: REST API Design, HTTP Verbs, Request/Response Patterns & Protocols Manifesto Level: 2–3 | Jonasan Type: Design Thinking Problem

What This Is

The manifesto maps Social Exchange Theory to "API interactions and data protocols as reciprocal human exchanges." This PR builds that into a learning module for REST API design. Social Exchange Theory (Blau, 1964) says relationships persist when exchange is perceived as fair and reciprocal. HTTP APIs are the most transactional version of this theory made digital: GET (I ask, you give), POST (I give, you acknowledge), DELETE (I request removal, you comply). The learner designs an API as a system of fair social exchange.

Trigger (Case Study):

You are designing the API for a community lending library — people lend their physical books to strangers for 2 weeks. Every transaction must be fair: - A borrower requests a book (GET) - The owner confirms availability (200 OK) or refuses (404 Not Found) - The borrower confirms the loan (POST) - The owner expects the book back on time (the contract) - If the borrower keeps the book too long, the trust score drops (business logic)

Design the complete REST API. What are the endpoints? What are the request bodies? What are the response codes? What happens when trust breaks down (unauthorized access, 401)?

A16 — "The Protein Blueprint — Biological Inheritance"

Python Concept: Object-Oriented Programming, Inheritance, Polymorphism & Abstract Base Classes Manifesto Level: 3 | Jonasan Type: Structured Inquiry

What This Is

Uses protein biology — how all proteins are built from the same 20 amino acids (base classes), assembled into structures (subclasses) that override general behavior with specific functions (polymorphism) — to teach OOP's inheritance hierarchy. DNA inheritance is literally the biological antecedent of class inheritance. The `__init__` method is gene expression. Method overriding is mutation and adaptation.

Trigger (Case Study):

You are modeling a biological research database of proteins. All proteins share certain properties: amino acid sequence, molecular weight, charge. But different protein families have specialized behaviors: Enzymes catalyze reactions. Transport proteins carry molecules. Receptor proteins bind signals.

Task 1: What code do you write ONCE that all protein types can share? Task 2: What behavior is common but expressed differently by each type (polymorphism)? Task 3: What if you want to ensure every protein type MUST implement a `function()` method — and raise an error if they don't?

Task 3 drives discovery of `abc.ABC` and `@abstractmethod`.

A17 — "The Musical Composer — Functional Programming"

Python Concept: Pure Functions, Higher-Order Functions, map/filter/reduce, Immutability Manifesto Level: 3 | Jonasan Type: Structured Inquiry

What This Is

Music composition — specifically the Indian classical raga structure — teaches functional programming principles. A raga is a set of rules (the scale, the allowed phrases, the time of day) that generates infinite variation without ever mutating the core rules. Pure functions are compositional: they take input, return output, and change nothing else. The composer of a raga doesn't modify notes — they compose them.

Trigger (Case Study):

A music analysis tool processes audio streams from 10,000 ragas. Each analysis step (normalize pitch, extract rhythm, identify scale, calculate emotional weight) is done by a different analyst.

Problem: Analyst 3 is modifying the pitch data before passing it to Analyst 4. Analyst 4's results are now wrong. But you can't find who corrupted the data — the chain has 8 steps.

How do you design the analysis pipeline so that NO step can modify the data it receives? Each step only returns a new, transformed copy.

This drives discovery of: immutable data (tuples over lists for data that must not change), `map()`, `filter()`, `functools.reduce()`, list comprehensions as functional expressions, and the concept that a function with no side effects is infinitely testable and debuggable.

A18 — "Anicca — The Context Manager"

Python Concept: Context Managers, `with` Statement, `__enter__`/`__exit__`, `contextlib`
Manifesto Level: 2–3 | **Jonasan Type:** Structured Inquiry

What This Is

Buddhist Anicca (impermanence) is used here for the specific concept of temporary, scoped existence — which is exactly what Python's `with` statement expresses. A file handle, a database connection, a mutex lock — these things must exist for a bounded time and then cease. The `with` statement is the Anicca principle in Python syntax: "this object exists in this context, and when the context ends, it is released regardless of what happens."

Trigger (Case Study):

A financial trading system opens thousands of database connections per second. A programmer writes:

```
conn = db.connect()
result = conn.execute(query) # If this raises an exception...
conn.close()                 # ...this line never runs
```

After 3 hours of trading, the system crashes. The database reports 10,000 open, unclosed connections.

How do you guarantee that a connection is always closed, even if an exception is raised in the middle?

Bonus: How do you write your own "safe resource" that other developers can use with the `with` statement?

This drives discovery of: `with` statement, `__enter__`/`__exit__` protocol, `contextlib.contextmanager` decorator, and `contextlib.suppress()`.

A19 — "The Fellowship's Dependency — Virtual Environments"

Python Concept: Virtual Environments, `pip`, `requirements.txt`, Package Conflicts
Manifesto Level: 1–2 | **Jonasan Type:** Structured Inquiry

What This Is

LOTR has a second mapping beyond Dijkstra's: the concept of a long quest where different allies need different tools and provisions — and mixing them causes conflicts. Legolas's arrows don't work with Gimli's axe-grip technique. Each Fellowship member has their own "dependency requirements." This teaches virtual environments: `venv`, `pip`, `requirements.txt`, and

dependency conflict resolution.

Trigger (Case Study):

You join a company and are handed two projects to maintain simultaneously: - Project Legolas requires `requests==2.25.0` and `numpy==1.19.0` - Project Gimli requires `requests==2.28.0` and `numpy==1.24.0`

If you install both in the same Python environment, one project breaks. You cannot install two versions of the same library in the same environment.

How do you work on both projects without them interfering with each other?

A20 — "The Color State Machine"

Python Concept: Enum, State Machines, Finite Automata & `dataclasses` Manifesto Level: 2–3 | Jonasan Type: Design Thinking Problem

What This Is

Uses Color Theory (listed in the mind map) to teach enumerations and state machines. Traffic lights are the canonical state machine, but pyBE's version uses color mixing theory — primary colors combine to create secondary colors through defined transitions — as the metaphor for finite state automaton transitions. The learner builds a state machine where states are colors and transitions are mixing operations.

Trigger (Case Study):

A paint-mixing robot arm at an art studio can hold one color of paint at a time. Its states are: Red, Blue, Yellow, Purple, Green, Orange, White, Black.

The rules: - Red + Blue → Purple - Blue + Yellow → Green - Red + Yellow → Orange - Any color + White → Tint of that color - From state Purple, the robot CANNOT directly go to Green. It must pass through Blue.

Task 1: Model this as a Python state machine. What data structure represents the states and allowed transitions? Task 2: Given a target color (Orange), compute the minimum sequence of mixing operations from a starting state (Blue).

This drives discovery of: Enum for states, dictionaries for transition tables, graph traversal for path finding, and `@dataclass` for state metadata.

A21 — "Piaget's Ladder — Type Systems and Cognitive Progression"

Python Concept: Python's Type System, Type Hints, mypy, Duck Typing & Protocol Manifesto Level: 3–4 | Jonasan Type: Design Thinking Problem

What This Is

Uses Piaget's Four Stages of Cognitive Development — sensorimotor, preoperational, concrete

operational, formal operational — as a map for Python's progression from duck typing (informal, intuitive) to Protocol-based structural typing (formal, abstract) to full generic type systems. A child in the sensorimotor stage doesn't classify objects by formal category — they interact with them directly. A Python function that accepts `obj.quack()` without checking type is doing the same thing.

Trigger (Case Study):

A junior developer writes a function that processes animals:

```
def make_sound(animal):  
    animal.speak()
```

This works for Duck, Dog, Cat. But when a developer passes a Car object (which also has a `.speak()` method — it "honks"), the function doesn't crash — and it shouldn't. But is this correct?

The team grows. 50 developers now call `make_sound()`. Some pass objects that have a `speak()` method that does the wrong thing. The function works but produces incorrect results.

How do you communicate to other developers exactly what kind of object `make_sound()` expects — without giving up Python's flexibility?

This drives discovery of: duck typing (stage 1), type hints with `isinstance()` (stage 2), `abc.ABC` abstract base classes (stage 3), and `typing.Protocol` for structural subtyping (stage 4 — the most Pythonic, most principled approach).

SECTION B — PLATFORM ENGINEERING PRs

These are technical infrastructure improvements to the pyBE platform itself. They make the existing codebase more powerful, reliable, and production-ready.

B1 — Real AI Integration

Replace Rule-Based Evaluator with Claude/OpenAI for Dynamic Scenario

Feedback Engineering Complexity: High | Priority: Critical

What This Is

The pyBE manifesto's learning engine currently uses deterministic, rule-based logic (keyword matching, scoring rubrics hardcoded in JSON) to evaluate learner responses. The README explicitly says "Later phases can replace those services with OpenAI, RAG, or TinyLLM components." This PR makes that leap: the evaluation engine becomes a real AI that reads the learner's natural language reasoning, understands context, detects misconceptions, and provides nuanced, adaptive feedback — not keyword matching.

What to Build (Full Specification)

Backend Changes:

```

server/src/services/
├── aiEvaluator.js          # New: calls Claude/OpenAI API with learner response
├── learningEngine.js      # Existing: add AI evaluation as fallback/primary
└── path
    ├── promptBuilder.js   # New: constructs the system prompt for the AI
    └── evaluator

```

The System Prompt Architecture (`promptBuilder.js`): The AI evaluator needs a structured system prompt that tells the model:

1. The scenario's expected Python construct (from `scenario.json`)
2. The theoretical anchor (so AI knows the domain context)
3. The scoring rubric (Theory 40%, Code 30%, Reflection 30%)
4. What "good reasoning" looks like for this scenario
5. What misconceptions to watch for

```

// promptBuilder.js
function buildEvaluatorPrompt(scenario, learnerResponse) {
  return {
    system: `You are a pyBE learning evaluator. You evaluate learner responses
    for the scenario: "${scenario.title}".

    Target Python construct: ${scenario.targetConstruct}
    Domain anchor: ${scenario.philosophicalAnchor}

    Scoring rubric:
    - Reasoning quality (0-40): Did the learner explain WHY they chose this
construct?
    - Code correctness (0-30): Is the implementation correct and idiomatic?
    - Reflection depth (0-30): Did the learner connect the solution to the
domain anchor?

    Common misconceptions to watch for: $
${JSON.stringify(scenario.commonMisconceptions)}

    Respond ONLY in this JSON format:
    {
      "scores": { "reasoning": 0-40, "code": 0-30, "reflection": 0-30 },
      "totalScore": 0-100,
      "misconceptionsDetected": [...],
      "feedbackByPillar": {
        "reasoning": "specific feedback on their reasoning",
        "code": "specific feedback on their code",
        "reflection": "specific feedback on their reflection"
      },
      "nextStepSuggestion": "what to explore next",
      "praisePoint": "one genuine specific thing they did well"
    },
    user: `Learner's response to the scenario:\n\n${learnerResponse}`
  };
}

```

API Route (`server/src/routes/evaluation.js`):

```

router.post('/evaluate', async (req, res) => {
  const { scenarioId, learnerResponse, sessionId } = req.body;
  const scenario = await ScenarioService.getById(scenarioId);
  const prompt = buildEvaluatorPrompt(scenario, learnerResponse);

```

```
const aiResponse = await callClaude(prompt); // or callOpenAI
const evaluation = JSON.parse(aiResponse);

await SessionService.saveEvaluation(sessionId, evaluation);
res.json(evaluation);
});
```

Conversational Hint System: When a learner clicks "Get a Hint," the AI doesn't give a generic hint — it reads what the learner has written so far and gives a contextual nudge:

```
function buildHintPrompt(scenario, currentAttempt, hintsGiven) {
  return {
    system: `You are a Socratic tutor for the pyBE scenario: "${scenario.title}"`,
    user: `The learner has written: "${currentAttempt}"`.
  };
  Never give the answer. Ask a question that nudges the learner one step closer.
  Hints given so far: ${hintsGiven}. Do not repeat these.`;
  user: `The learner has written: "${currentAttempt}"`.
  They are stuck. Give them ONE Socratic question (not an answer) that nudges them toward discovering: ${scenario.targetConstruct}`;
};
```

Environment Setup:

- Add ANTHROPIC_API_KEY or OPENAI_API_KEY to .env
- Add anthropic or openai npm package
- Add a feature flag USE_AI_EVALUATION=true in .env so it falls back to rule-based if the key is absent (backwards compatibility)

Frontend Changes:

- Update src/components/EvaluationResult.jsx to render the new structured JSON (scores by pillar, misconceptions, next steps)
- Add a streaming display so feedback appears word by word (better UX)
- Add a "Get AI Hint" button that calls the hint endpoint

Files to Create/Modify:

server/src/services/aiEvaluator.js	CREATE
server/src/services/promptBuilder.js	CREATE
server/src/routes/evaluation.js	MODIFY (add AI path)
server/.env.example	MODIFY (add AI keys)
client/src/components/EvaluationResult.jsx	MODIFY
client/src/components/HintButton.jsx	CREATE

B2 — Live Python Execution Sandbox (Pyodide)

In-Browser Python Interpreter with Live Output Engineering Complexity: Medium | Priority: High

What This Is

Embed Pyodide (Python compiled to WebAssembly) directly in the React frontend so that after a

learner completes a scenario's reasoning phase, they can write and run actual Python code in-browser and see real output — without any backend, without any security concerns. The sandbox is completely client-side.

What to Build

Installation:

```
# No npm install needed – Pyodide loads from CDN in the browser
# Add to index.html or load dynamically in the component
```

The Python Sandbox Component (client/src/components/PythonSandbox.jsx):

```
import { useState, useEffect } from 'react';

export function PythonSandbox({ starterCode, scenarioId }) {
  const [pyodide, setPyodide] = useState(null);
  const [code, setCode] = useState(starterCode || '# Write your Python here\n');
  const [output, setOutput] = useState('');
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    async function loadPyodide() {
      // Load Pyodide from CDN
      const pyodideInstance = await window.loadPyodide({
        indexURL: "https://cdn.jsdelivr.net/pyodide/v0.25.0/full/",
        stdout: (text) => setOutput(prev => prev + text + '\n'),
        stderr: (text) => setOutput(prev => prev + '[ERROR] ' + text + '\n'),
      });
      setPyodide(pyodideInstance);
      setIsLoading(false);
    }
    loadPyodide();
  }, []);

  async function runCode() {
    setOutput('');
    try {
      await pyodide.runPythonAsync(code);
    } catch (err) {
      setOutput('[Python Error]\n' + err.message);
    }
  }

  return (
    <div className="sandbox-container">
      {isLoading && <div className="loading">Loading Python runtime...</div>}
      <textarea
        value={code}
        onChange={e => setCode(e.target.value)}
        className="code-editor"
        rows={15}
        spellCheck={false}
      />
      <button onClick={runCode} disabled={isLoading}>> Run</button>
      <pre className="output-panel">{output || 'Output will appear here'}</pre>
    </div>
  );
}
```

Integration into Scenario Flow:

- The sandbox appears at Stage 5 of the learner flow (after reasoning is submitted and scored)
- `starterCode` is pre-populated from `scenario.solution.starterTemplate` (a guided scaffold, not the full solution)
- If the learner's code produces a `NameError` or `TypeError`, the AI hint system is triggered automatically with the error context
- Add `tracemalloc` support: after running, show memory used by the code

Security: Pyodide runs entirely in the browser's JavaScript sandbox. No code reaches the server. No injection risk.

Files to Create:

<code>client/src/components/PythonSandbox.jsx</code>	CREATE
<code>client/src/components/OutputPanel.jsx</code>	CREATE
<code>client/index.html</code>	MODIFY (add Pyodide CDN script)
<code>client/src/pages/ScenarioDetail.jsx</code>	MODIFY (embed sandbox)

B3 — Authentication System (JWT + User Profiles)

Full Auth Flow: Registration, Login, Protected Routes, Per-User Data Engineering

Complexity: High | Priority: High

What This Is

The repo README explicitly states "no login flow for now." This is the V1 feature that unlocks everything else: per-user progress tracking, leaderboards, contributor credit, and community features. This PR adds a complete JWT-based authentication system.

What to Build

Backend (Node.js/Express):

Install packages:

```
npm install bcryptjs jsonwebtoken express-validator
```

User Model (server/src/models/user.json or MongoDB schema):

```
{
  "id": "uuid",
  "username": "string (unique)",
  "email": "string (unique)",
  "passwordHash": "string (bcrypt, 12 rounds)",
  "createdAt": "ISO timestamp",
  "level": 1,
  "totalScore": 0,
  "domainPreferences": [],
  "contributorBadges": [],
  "scenarioAttempts": {}
}
```

Auth Routes (server/src/routes/auth.js):

```
// POST /api/auth/register
```

```

router.post('/register', async (req, res) => {
  const { username, email, password } = req.body;
  const passwordHash = await bcrypt.hash(password, 12);
  const user = await UserService.create({ username, email, passwordHash });
  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET,
{ expiresIn: '7d' });
  res.json({ token, user: { id: user.id, username: user.username } });
});

// POST /api/auth/login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await UserService.findByEmail(email);
  if (!user || !await bcrypt.compare(password, user.passwordHash)) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }
  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET,
{ expiresIn: '7d' });
  res.json({ token, user: { id: user.id, username: user.username } });
});

```

Auth Middleware (server/src/middleware/authenticate.js):

```

module.exports = function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
};

```

Frontend (client/src/):

client/src/	
├ contexts/AuthContext.jsx	CREATE – React context for auth state
├ pages/Login.jsx	CREATE
├ pages/Register.jsx	CREATE
├ components/ProtectedRoute.jsx	CREATE – wrapper for protected pages
└ utils/api.js	MODIFY – add JWT header to all requests

AuthContext pattern:

```

export const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    // Restore from localStorage on reload
    const token = localStorage.getItem('pybe_token');
    return token ? jwtDecode(token) : null;
  });

  const login = async (email, password) => { /* ... */ };
  const logout = () => { localStorage.removeItem('pybe_token');
setUser(null); };

  return <AuthContext.Provider value={{ user, login, logout
}}>{children}</AuthContext.Provider>;
}

```

B4 — MongoDB Migration

From JSON File Storage to Mongoose + MongoDB Atlas Engineering Complexity: Medium | Priority: High (required before scaling)

What to Build

Install:

```
npm install mongoose dotenv
```

Mongoose Schemas (server/src/models/):

Scenario.js:

```
const scenarioSchema = new mongoose.Schema({
  title: { type: String, required: true },
  domain: String,
  philosophicalAnchor: String,
  pythonConcept: String,
  difficultyLevel: { type: Number, min: 1, max: 5 },
  jonsasanType: { type: String, enum: ['Structured Inquiry', 'Design Thinking', 'Dilemma'] },
  casestudy: String,
  hints: [{ order: Number, text: String }],
  expectedConstructs: [String],
  scoringRubric: { reasoning: Number, code: Number, reflection: Number },
  reflectionPrompts: [String],
  tags: [String],
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  isApproved: { type: Boolean, default: false },
  approvedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
});
```

Session.js:

```
const sessionSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  scenarioId: { type: mongoose.Schema.Types.ObjectId, ref: 'Scenario', required: true },
  attemptNumber: Number,
  learnerResponse: String,
  scores: { reasoning: Number, code: Number, reflection: Number, total: Number },
  misconceptionsDetected: [String],
  hintsUsed: Number,
  timeSpentSeconds: Number,
  completedAt: Date
});
```

Migration Script (server/scripts/migrate-json-to-mongo.js):

```
const seedScenarios = require('../src/data/scenarios.json');
await Scenario.insertMany(seedScenarios.map(transformToSchema));
console.log(`Migrated ${seedScenarios.length} scenarios`);
```

Environment toggle: Add STORAGE_MODE=json (default) or STORAGE_MODE=mongodb in .env so the app still works without MongoDB for local development.

B5 — WebSocket Real-Time Feedback

Live AI Response Streaming as Learner Types Engineering Complexity: High | Priority: Medium

What to Build

Install:

```
npm install socket.io # server
npm install socket.io-client # client
```

Server-side (server/src/sockets/evaluationSocket.js):

```
io.on('connection', (socket) => {
  socket.on('START_EVALUATION', async ({ scenarioId, response, sessionId }) => {
    socket.emit('EVALUATION_STATUS', { status: 'thinking' });

    // Stream Claude's response token by token
    const stream = await streamClaudeEvaluation(scenarioId, response);
    for await (const chunk of stream) {
      socket.emit('EVALUATION_CHUNK', { text: chunk });
    }
    socket.emit('EVALUATION_COMPLETE', { sessionId });
  });

  socket.on('TYPING_UPDATE', async ({ scenarioId, currentText }) => {
    // Live confidence meter – rough heuristic evaluation as learner types
    const confidence = computeRoughConfidence(scenarioId, currentText);
    socket.emit('CONFIDENCE_UPDATE', { confidence }); // 0-100
  });
});
```

Frontend — Live Confidence Meter: As the learner types their response, a thin colored bar at the bottom of the text area fills left-to-right from red to green, updating every 1.5 seconds. This is a rough heuristic (keyword presence, response length, mention of domain anchor), not the final AI score. When the learner submits, the real AI evaluation begins and the meter is replaced with the full breakdown.

B6 — Spaced Repetition System (SM-2 Algorithm)

Scenario Review Scheduler Based on Forgetting Curve Engineering Complexity: Medium | Priority: Medium

What to Build

The SM-2 algorithm (used by Anki) schedules each scenario for review based on how well the learner did and how long ago they last saw it. Scenarios with high scores get longer intervals; scenarios with low scores come back sooner.

Backend Service (server/src/services/spacedRepetition.js):

```
function calculateNextReview(lastScore, currentInterval, easeFactor) {
  // SM-2 algorithm
  const quality = Math.round((lastScore / 100) * 5); // convert to 0-5 scale
  let newInterval;
```

```

if (quality < 3) {
  newInterval = 1; // Review tomorrow – learner struggled
} else if (currentInterval === 0) {
  newInterval = 1;
} else if (currentInterval === 1) {
  newInterval = 6;
} else {
  newInterval = Math.round(currentInterval * easeFactor);
}

const newEaseFactor = Math.max(1.3,
  easeFactor + 0.1 - (5 - quality) * (0.08 + (5 - quality) * 0.02)
);

const nextReviewDate = new Date();
nextReviewDate.setDate(nextReviewDate.getDate() + newInterval);

return { nextReviewDate, newInterval, newEaseFactor };
}

```

Dashboard Feature — "Due Today" Panel: Shows learners which scenarios are scheduled for review today. Sorted by urgency (overdue first). Each card shows when it was last attempted and the score from that attempt.

Migration: Add `nextReviewDate`, `reviewInterval`, `easeFactor`, `lastAttemptScore` fields to the user-scenario relationship record.

B7 — Full Test Suite + CI Pipeline

Jest, React Testing Library, Supertest, GitHub Actions Engineering Complexity: Medium | Priority: High (code quality signal)

What to Build

Install:

```

npm install --save-dev jest @testing-library/react @testing-library/jest-dom
supertest

```

Unit Tests (server/src/__tests__/):

```

// learningEngine.test.js
describe('LearningEngine', () => {
  test('scores high-quality reasoning appropriately', () => {
    const result = evaluateResponse(mockScenario, highQualityResponse);
    expect(result.scores.reasoning).toBeGreaterThan(30);
  });

  test('detects common misconceptions', () => {
    const result = evaluateResponse(mockScenario, misconceptionResponse);
    expect(result.misconceptionsDetected).toContain('confused_scope');
  });
});

```

API Tests (server/src/__tests__/routes/):

```

// scenarios.test.js
const request = require('supertest');

```

```
const app = require('../..../app');

describe('GET /api/scenarios', () => {
  test('returns scenario list', async () => {
    const res = await request(app).get('/api/scenarios');
    expect(res.status).toBe(200);
    expect(res.body).toHaveLength('scenarios');
    expect(Array.isArray(res.body.scenarios)).toBe(true);
  });
});
```

Component Tests (client/src/__tests__/):

```
// ScenarioCard.test.jsx
import { render, screen } from '@testing-library/react';
import ScenarioCard from '../components/ScenarioCard';

test('displays difficulty level badge', () => {
  render(<ScenarioCard scenario={mockScenario} />);
  expect(screen.getByText('Level 3')).toBeInTheDocument();
});
```

GitHub Actions (.github/workflows/ci.yml):

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: { node-version: '18' }
      - run: cd server && npm ci && npm test
      - run: cd client && npm ci && npm test
```

This CI pipeline means every PR gets automatically tested. A failing test blocks merge. This signals enormous engineering maturity.

B8 — PDF Learning Report Generator

Per-Session and Weekly Learning Reports Engineering Complexity: Low-Medium | Priority: Low-Medium

What to Build

Option 1 — Client-Side (jsPDF):

```
npm install jspdf html2canvas # client-side
```

Generates a PDF entirely in the browser from the dashboard DOM. Fast, no backend needed.

Option 2 — Server-Side (Puppeteer):

```
npm install puppeteer # server-side
```

Generates a PDF by rendering an HTML template to a headless browser. Higher quality, more control.

Report Contents:

- Learner name, date range
- Scenarios attempted (with scores per pillar)
- Concepts mastered (green checkmarks), in progress (yellow), not started (grey)
- Misconceptions detected with explanations
- Domains explored (the "from where" map)
- Top 3 reflection responses
- Next recommended scenarios

API Route:

```
router.get('/api/users/:id/report', authenticate, async (req, res) => {
  const report = await generatePDF(req.params.id, req.query.from, req.query.to);
  res.setHeader('Content-Type', 'application/pdf');
  res.setHeader('Content-Disposition', 'attachment; filename=pybe-report.pdf');
  res.send(report);
});
```

SECTION C — PLATFORM & COMMUNITY FEATURE PRs

These are product-level features that transform pyBE from a solo learning app into a living ecosystem. They operationalize Social Exchange Theory, enable community contribution, and add the mentorship layer.

C1 — The Anti-Superficiality Checker

AI-Powered PR Quality Evaluator Engineering Complexity: Medium | Priority: HIGHEST
— This is the Most Strategic PR

What This Is

The manifesto says "I will not be easy to satisfy" and defines a 4-pillar checklist for every PR. This PR automates that checklist using the Claude API. When a contributor submits a PR to pyBE, a GitHub Action triggers an AI evaluation that reads the PR description and scores it against all four pillars. PRs that score below 60 receive specific, actionable feedback on which pillars are weak. This PR is the most philosophically aligned with the manifesto — you are operationalizing the mentor's own standards into code.

What to Build

GitHub Action (`.github/workflows/pr-checker.yml`):

```
name: Anti-Superficiality Check
on: [pull_request]
jobs:
  evaluate-pr:
    runs-on: ubuntu-latest
```

```

steps:
  - name: Check PR Quality
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
      PR_BODY: ${ github.event.pull_request.body }
    run: node scripts/check-pr-quality.js

```

PR Quality Evaluator (scripts/check-pr-quality.js):

```

const response = await fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'x-api-key':
process.env.ANTHROPIC_API_KEY },
  body: JSON.stringify({
    model: 'claude-sonnet-4-6',
    max_tokens: 1000,
    messages: [{
      role: 'user',
      content: `You are the pyBE PR quality checker. Evaluate this PR against
the 4-pillar manifesto.

```

PR CONTENT:
\${process.env.PR_BODY}

Score each pillar 0-25:

1. Theoretical/Philosophical Justification: Does it have a rigorous "Why"?
2. Interdisciplinary Mapping: Is the non-programming domain anchor clear and meaningful?
3. Case Study / Trigger: Is there a real problem scenario that leads to discovery?
4. Software Engineering Reality: Is there genuine technical depth (hardware/firmware/memory)?

Respond ONLY in JSON:

```

{
  "scores": { "theory": 0-25, "anchor": 0-25, "trigger": 0-25, "reality": 0-
25 },
  "total": 0-100,
  "passed": true/false,
  "pillarFeedback": {
    "theory": "specific feedback",
    "anchor": "specific feedback",
    "trigger": "specific feedback",
    "reality": "specific feedback"
  },
  "verdict": "one sentence summary"
}
`
  })
})
});

```

```

const evaluation = JSON.parse(result.content[0].text);
if (!evaluation.passed) {
  // Post comment to PR via GitHub API
  await postPRComment(formatFeedback(evaluation));
  process.exit(1); // Fail the CI check
}

```

Output: Every PR gets an automatic comment showing:

```
## pyBE Anti-Superficiality Check Results
```

Pillar	Score	Maximum
--------	-------	---------

```
|-----|-----|-----|
| Theoretical Justification | 22 | 25 |
| Interdisciplinary Mapping | 18 | 25 |
| Case Study / Trigger | 20 | 25 |
| Engineering Reality | 12 | 25 |
| **TOTAL** | **72/100** | □ PASSED |
```

****Theory:**** Strong philosophical grounding. The Panchatantra connection is well-argued.

****Reality:**** Needs more hardware depth. Consider demonstrating the metaclass in the context of a real ORM or audit system.

C2 — The Domain Mapper (Rhizomatic Path Visualizer)

D3.js Force-Directed Graph of Each Learner's Personal Learning Network Engineering

Complexity: Medium-High | Priority: High

What This Is

The manifesto explicitly states the learning model is "rhizomatic" — no single correct path, a network where every entry point is valid. This PR makes that network visible. Each learner gets a personal D3.js force-directed graph showing: which domains they entered from (music, biology, folklore), which Python concepts they discovered, and the connections between them. Two learners who learned recursion — one through DNA folding, one through Panchatantra — will have completely different-looking graphs. This makes the manifesto's philosophy literally visible.

What to Build

D3.js Component (client/src/components/DomainMapper.jsx): The graph has three node types:

- **Domain nodes** (large, colored by domain: Biology=green, Music=orange, Folklore=gold) — where the learner entered
- **Concept nodes** (medium, blue) — Python constructs discovered
- **Connection edges** — labeled with the scenario that connected domain to concept

```
import * as d3 from 'd3';
import { useEffect, useRef } from 'react';

export function DomainMapper({ learnerGraph }) {
  const svgRef = useRef();

  useEffect(() => {
    const svg = d3.select(svgRef.current);
    const { nodes, links } = learnerGraph;

    const simulation = d3.forceSimulation(nodes)
      .force('link', d3.forceLink(links).id(d => d.id).distance(120))
      .force('charge', d3.forceManyBody().strength(-300))
      .force('center', d3.forceCenter(400, 300));

    const link = svg.append('g').selectAll('line')
      .data(links).enter().append('line')
      .style('stroke', '#aaa').style('stroke-width', 2);
```

```

const node = svg.append('g').selectAll('circle')
  .data(nodes).enter().append('circle')
  .attr('r', d => d.type === 'domain' ? 20 : 12)
  .attr('fill', d => d.type === 'domain' ? domainColors[d.domain] :
'#4A90D9')
  .call(d3.drag() // Make nodes draggable
    .on('start', dragStart)
    .on('drag', dragged)
    .on('end', dragEnd));

simulation.on('tick', () => {
  link.attr('x1', d => d.source.x).attr('y1', d => d.source.y)
    .attr('x2', d => d.target.x).attr('y2', d => d.target.y);
  node.attr('cx', d => d.x).attr('cy', d => d.y);
});
}, [learnerGraph]);

return <svg ref={svgRef} width={800} height={600} />;
}

```

Backend — Graph Data Endpoint:

```

// GET /api/users/:id/learning-graph
router.get('/learning-graph', authenticate, async (req, res) => {
  const sessions = await Session.find({ userId:
req.user.id }).populate('scenarioId');

  const nodes = new Map();
  const links = [];

  for (const session of sessions) {
    const scenario = session.scenarioId;

    // Add domain node
    if (!nodes.has(scenario.domain)) {
      nodes.set(scenario.domain, { id: scenario.domain, type: 'domain', domain:
scenario.domain });
    }

    // Add concept node
    if (!nodes.has(scenario.pythonConcept)) {
      nodes.set(scenario.pythonConcept, { id: scenario.pythonConcept, type:
'concept' });
    }

    // Add link from domain to concept
    links.push({
      source: scenario.domain,
      target: scenario.pythonConcept,
      scenarioTitle: scenario.title,
      score: session.scores.total
    });
  }

  res.json({ nodes: Array.from(nodes.values()), links });
});

```

C3 — Scenario Builder Admin Panel

UI for Creating New Scenarios Without Touching JSON Files Engineering Complexity:

Medium | Priority: High

What to Build

Protected Route: Only accessible to users with `role: 'admin'` or `role: 'contributor'` (contributor can create but not publish — admin reviews first).

Multi-Step Form (`client/src/pages/ScenarioBuilder.jsx`):

Step 1: Foundation

- Title, domain anchor, theoretical framework
- Python concept being taught
- Difficulty level (1–5 slider with descriptors)
- Jonasan taxonomy type (dropdown with descriptions of each type)

Step 2: The Trigger

- Case study narrative (rich text editor, Markdown supported)
- Problem constraints
- What constructs the learner should discover (multi-select from master list)

Step 3: Hints & Reflection

- 5 hint slots (ordered, must be Socratic questions — the form validates no hint directly gives the answer)
- 4 reflection prompts

Step 4: Solution & Scoring

- Reference solution code (code editor with syntax highlighting)
- Starter template code (what the learner sees)
- Scoring rubric weights (3 sliders that must sum to 100)

Step 5: Preview

- Full preview of how the scenario will look to a learner
- Preview of how it will be evaluated

Step 6: Submit for Review

- Saves with `status: 'pending_review'`
- Notifies admin via email

C4 — Peer Discussion Threads

Community Discourse Per Scenario Engineering Complexity: Medium | Priority: Medium

What to Build

Each scenario gets a discussion section with threaded comments. After completing a scenario, learners can share their reasoning approach and see how others tackled it.

Comment Schema:

```
{
  scenarioId: ObjectId,
  authorId: ObjectId,
  content: String,           // Their reasoning approach
  pythonConstruct: String,   // Which construct they used (tagged, not free text)
  domainConnection: String, // Which non-programming domain it clicked for them
  upvotes: [ObjectId],      // User IDs who upvoted
  isAccepted: Boolean,      // Mentor-marked as exemplary
  createdAt: Date
}
```

Key Design Decision: Comments are not anonymous. Every comment shows:

- The commenter's username
- Their current pyBE level
- Which domain they came from (their background)

This makes Social Exchange Theory visible: "A musician found recursion through rhythm. A biology student found it through DNA. A folklore enthusiast found it through Panchatantra. All three are now discussing the same Python concept from three different perspectives."

C5 — Gamification Engine

XP, Levels, Badges, Multi-Category Leaderboard Engineering Complexity: Medium | Priority: High

What to Build

XP Calculation Logic (server/src/services/gamification.js):

```
function calculateXP(session) {
  const baseXP = session.scores.total;           // 0-100 base
  const hintPenalty = session.hintsUsed * 5;     // -5 XP per hint used
  const speedBonus = session.timeSpentSeconds < 600 ? 10 : 0; // +10 for under
10 min
  const reflectionBonus = session.scores.reflection > 25 ? 15 : 0; // +15 for
excellent reflection
  const firstAttemptBonus = session.attemptNumber === 1 ? 20 : 0; // +20 for
first try

  return Math.max(0, baseXP - hintPenalty + speedBonus + reflectionBonus +
firstAttemptBonus);
}
```

Level Thresholds:

```
const LEVELS = [
  { level: 1, name: 'Discoverer', minXP: 0, maxXP: 200, color: '#78C257' },
  { level: 2, name: 'Apprentice', minXP: 200, maxXP: 600, color: '#4A90D9' },
  { level: 3, name: 'Reasoner', minXP: 600, maxXP: 1400, color: '#9B59B6' },
]
```

```
{ level: 4, name: 'Architect', minXP: 1400, maxXP: 3000, color: '#E67E22' },
{ level: 5, name: 'Pythonista', minXP: 3000, maxXP: Infinity, color: '#E74C3C'
}
];
```

Badges (aligned with manifesto domains):

```
const BADGES = {
  'first_blood': { name: 'First Blood', trigger: 'first scenario completed',
    icon: '☐' },
  'folklore_explorer': { name: 'Panchatantra Path', trigger: 'completed a
    Folklore scenario', icon: '☐' },
  'biologist': { name: 'Protein Thinker', trigger: 'completed a Biology
    scenario', icon: '☐' },
  'musician': { name: 'Taal Master', trigger: 'completed a Music scenario',
    icon: '☐' },
  'philosopher': { name: 'Anicca Seeker', trigger: 'completed a Philosophy
    scenario', icon: 'ॐ' },
  'hardware_toucher': { name: 'Level 5 Reached', trigger: 'completed a Level 5
    scenario', icon: '⚡' },
  'socratic_thinker': { name: 'No Hints Needed', trigger: '5 scenarios completed
    without hints', icon: '☐' },
  'manifesto_reader': { name: 'Manifesto Internalized', trigger: 'submitted a PR
    that passes anti-superficiality check', icon: '☐' }
};
```

Multi-Category Leaderboard (not just "top scorers" — that discourages beginners):

- Weekly Progress (XP gained this week)
- Concept Mastery (breadth of concepts covered)
- Best Reasoning (highest reasoning sub-scores)
- Most Helpful Contributor (upvotes received on discussion posts)
- Domain Pioneer (first person to complete a scenario in each domain)

C6 — Mentor Review Dashboard

PR Approval Workflow for Scenario Contributions Engineering Complexity: Medium | Priority: High (needed for community PRs)

What to Build

Admin-Only Route (/admin/review):

Shows a queue of all pending scenario submissions with:

- Full scenario preview (exactly how learners will see it)
- Manifesto compliance checklist (auto-filled by Anti-Superficiality Checker if active)
- Mentor's private comment box
- Action buttons: Approve / Request Changes / Reject

Approval Schema:

```
{
```

```

scenarioId: ObjectId,
reviewerId: ObjectId,
status: { type: String, enum: ['pending', 'approved', 'needs_changes',
'rejected'] },
antiSuperficialityScore: Number,
mentorComments: String,
changeRequests: [String],    // List of specific changes needed
reviewedAt: Date
}

```

Notification System: When a scenario is approved/rejected, the contributor receives an in-app notification with the mentor's feedback. Approved scenarios are automatically promoted to status: 'published' and appear in the scenario browser.

C7 — Learning Analytics Dashboard

Radar Chart, Learning Velocity, Misconception Heatmap Engineering Complexity: Medium | Priority: High

What to Build (replace existing static dashboard)

Component 1 — Concept Mastery Radar Chart: A spider/radar chart showing proficiency across all Python concept areas. Built with Recharts or Chart.js.

```

import { RadarChart, PolarGrid, PolarAngleAxis, Radar, ResponsiveContainer }
from 'recharts';

const conceptData = [
  { concept: 'Data Structures', score: 82 },
  { concept: 'Recursion', score: 45 },
  { concept: 'OOP', score: 67 },
  { concept: 'Concurrency', score: 23 },
  { concept: 'Memory Management', score: 58 },
  { concept: 'Functional', score: 71 },
];

<RadarChart data={conceptData}>
  <PolarGrid />
  <PolarAngleAxis dataKey="concept" />
  <Radar dataKey="score" fill="#4A90D9" fillOpacity={0.6} />
</RadarChart>

```

Component 2 — Learning Velocity Graph: A time-series line chart showing average score per week over the last 12 weeks. Shows if the learner is improving, plateauing, or regressing.

Component 3 — Misconception Heatmap: A 2D grid where rows are Python concepts and columns are weeks. Cell color intensity = frequency of that misconception that week. Makes recurring misunderstandings immediately visible.

Component 4 — Domain Entry Map: Shows which non-programming domains the learner has explored. Displayed as colored bubbles sized by number of scenarios completed in that domain.

Component 5 — Streak & Consistency: Days active, current streak, longest streak. Calendar heatmap (GitHub-style) showing days with activity.

C8 — Dark Mode + Mobile Responsive Redesign

Color Theory Applied to the UI — Philosophical Alignment with the Manifesto Engineering Complexity: Low-Medium | Priority: Medium

What to Build

The Color Theory module (A20) teaches that color carries information. This PR applies that principle to pyBE's own UI: every visual element has a semantic meaning, not just decorative purpose.

CSS Variable System:

```
:root {
  /* Semantic color tokens – aligned with pyBE level system */
  --color-discovery: #78C257;      /* Level 1: calm, approachable green */
  --color-growth: #4A90D9;         /* Level 2-3: trustworthy blue */
  --color-mastery: #9B59B6;        /* Level 4: deep purple – complexity */
  --color-hardware: #E74C3C;       /* Level 5: red – power, hardware */

  /* Domain colors */
  --domain-biology: #27AE60;
  --domain-music: #E67E22;
  --domain-folklore: #F39C12;
  --domain-philosophy: #8E44AD;
  --domain-literature: #2980B9;

  /* Light mode base */
  --bg-primary: #FAFAF7;
  --bg-secondary: #F0EDE6;
  --text-primary: #1A1A2E;
  --text-secondary: #4A4A6A;
  --border: #D5D0C5;
}

[data-theme="dark"] {
  --bg-primary: #0F0F1A;
  --bg-secondary: #1A1A2E;
  --text-primary: #E8E6F0;
  --text-secondary: #9B99A8;
  --border: #2A2A3E;
}
```

Theme Toggle Component: Stores preference in `localStorage`. Defaults to system preference (`prefers-color-scheme`).

Mobile Responsive Breakpoints:

- 320px: Single column, no sidebar, hamburger menu
- 768px: Two columns, sidebar collapses to icons
- 1024px+: Full layout with side panel

C9 — Contributor Credit System

Social Exchange Theory Operationalized as a Platform Feature Engineering Complexity: Medium | Priority: Medium

What This Is

The manifesto says Social Exchange Theory justifies community contribution. Every act of giving to the repository should create visible, lasting credit for the contributor. This PR makes that visible at the platform level.

What to Build

Contributor Profile Page (/contributors/:username):

- All approved scenarios they authored (with their names permanently attached)
- Aggregate learner impact: how many learners have completed their scenarios, average score, most common misconception their scenario resolved
- Domain badges earned as a contributor
- GitHub-style contribution heatmap showing PR activity over time
- Quote from their best PR's philosophical anchor (shown as a decorative element)

"Created by" Attribution on Scenario Cards: Every scenario card shows the contributor's avatar and name. This is their permanent intellectual property mark.

Contribution Leaderboard (separate from learner leaderboard):

- Most scenarios approved
 - Most impactful scenario (learners who improved most after it)
 - Most creative domain mapping (community upvotes on novelty)
 - Most Pythonic solution (peer-voted code quality)
-

C10 — Case Study Template Enforcer

Standardized PR Submission Form + Validation Engineering Complexity: Low | Priority: High (foundational for community)

What to Build

A JSON schema and validation layer that enforces the structure of every new scenario submitted. Without this, contributors write inconsistent, incomplete scenarios.

Scenario Schema Validation (server/src/validation/scenarioSchema.js):

```
const Joi = require('joi');

const scenarioSchema = Joi.object({
  title: Joi.string().min(10).max(100).required(),
  domain: Joi.string().valid(
    'Biology', 'Music', 'Folklore/Panchatantra', 'Literature',
    'Philosophy/Buddhism', 'Pop Culture/Harry Potter', 'Pop Culture/LOTR',
    'Pop Culture/Avengers', 'Physics', 'Psychology', 'Linguistics',
    'Culinary', 'Sports', 'Other'
  ).required(),
  philosophicalAnchor: Joi.string().min(50).required()
    .messages({ 'string.min': 'Theoretical anchor must be at least 50
```

```

characters. Explain the WHY deeply.' }},
  interdisciplinaryMapping: Joi.string().min(30).required(),
  caseStudy: Joi.string().min(100).required()
    .messages({ 'string.min': 'Case study must be at least 100 characters. Write
a full scenario.' }},
  jonsasanType: Joi.string().valid('Structured Inquiry', 'Design Thinking',
'Dilemma').required(),
  targetConstructs: Joi.array().items(Joi.string()).min(1).max(5).required(),
  hints: Joi.array().items(
    Joi.object({
      order: Joi.number().integer().min(1).max(5).required(),
      text: Joi.string().min(20).required()
    })
  ).min(3).max(5).required(),
  scoringRubric: Joi.object({
    reasoning: Joi.number().min(30).max(50).required(),
    code: Joi.number().min(20).max(40).required(),
    reflection: Joi.number().min(20).max(40).required()
  }).required(),
  reflectionPrompts:
Joi.array().items(Joi.string().min(20)).min(3).max(5).required(),
  engineeringRealityLevel: Joi.number().min(1).max(5).required(),
  engineeringRealityDescription: Joi.string().min(50).required()
    .messages({ 'string.min': 'Describe the real engineering depth. Hardware?
Memory? Firmware? Make it specific.' })
});

```

PR Template (.github/PULL_REQUEST_TEMPLATE.md): When a contributor opens a PR, this template auto-fills:

```

## 📖 Theoretical/Philosophical Justification
<!-- Minimum 2 paragraphs. Explain the WHY. What theory, science, or philosophy
grounds this? -->

## 🗎 Interdisciplinary Mapping
<!-- Format: [Domain] → [Concept/Theory] → [Python Construct] -->

## 📖 Case Study / Trigger
<!-- The full problem scenario. Second person, present tense. NO syntax hints.
-->

**Jonasan Taxonomy Type:** [ ] Structured Inquiry [ ] Design Thinking [ ]
Dilemma

## ⚙️ Software Engineering Reality
<!-- What is the actual technical depth? Hardware? Memory optimization?
Firmware? System calls? -->

**Level on pyBE Scale (1-5):** ____

## 🗎 Self-Checklist
- [ ] I have not given any syntax hints in the case study
- [ ] The theoretical anchor is at least 2 paragraphs
- [ ] The engineering reality goes beyond "it works"
- [ ] I would not be embarrassed if my mentor read this

```

IMPLEMENTATION PRIORITY ORDER

Recommended sequence for maximum impact

Priority	PR	Why First
1	C10 — Case Study Template Enforcer	Foundational. Every other PR needs this structure.
2	A1 — The Panchatantra Mirror	First to claim Folklore. Philosophically tight. Shows manifesto mastery.
3	C1 — Anti-Superficiality Checker	Shows you operationalized the mentor's own standards into code. Unforgettable.
4	A2 — DNA Helix Recursion	Unclaimed Biology domain. Deep engineering (tracemalloc).
5	B1 — Real AI Integration	Makes the core experience 10x better. README explicitly invites this.
6	B2 — Pyodide Sandbox	Zero infra cost, massive "wow" factor, pure frontend creativity.
7	A7 — Maslow's Stack	Achieves Level 5 (hardware touch). Most ambitious content PR.
8	C2 — Domain Mapper	Makes the rhizomatic philosophy literally visible.
9	B3 — Authentication	Unlocks everything else (per-user data, leaderboards, contributions).
10	C5 — Gamification Engine	Engagement layer. Needs auth first.
11–31	All others	In order listed in their sections

Total: 31 unique, fully-specified ideas. 21 philosophical content PRs + 8 platform engineering PRs + 10 community/feature PRs (counting C10). All aligned with the pyBE Manifesto.